



SWEN 221

A surreal, dreamlike landscape. In the center, a person stands on a small, dark, rocky outcrop that appears to be part of a vast, ethereal sea of clouds. The person is looking out over a vast, colorful sky. The sky is filled with large, billowing clouds in shades of pink, purple, blue, and yellow. A bright, glowing light source, possibly a sun or moon, is visible in the distance, casting a warm glow over the scene. The sky is also filled with stars, a comet streaking across the upper left, and a large, colorful nebula or galaxy in the upper right. The overall atmosphere is one of wonder and exploration.

Marco Servetto
CO258

www.youtube.com/MarcoServetto



Exceptions



Error Handling (wrong way)

```
/**  
if n >= 0 return n! else return -1  
*/  
public static int factorial(int n){  
    if(n < 0){ return -1; }//represents an error condition  
    if(n == 0){ return 1; }  
    return n * factorial(n - 1);  
}
```

Error Handling (wrong way)

```
/**  
if n >= 0 return n! else return -1  
*/  
public static int factorial(int n){  
    if(n < 0){ return -1; }//represents an error condition  
    if(n == 0){ return 1; }  
    return n * factorial(n - 1);  
}
```

- Pre-exception solution. Still used in some situations...
 - But, what about `Object ArrayList.get(int index)`?

Error Handling (wrong way)

```
/**
if n >= 0 return n! else return -1
*/
public static int factorial(int n){
    if(n < 0){ return -1; }//represents an error condition
    if(n == 0){ return 1; }
    return n * factorial(n - 1);
}
```

- Pre-exception solution. Still used in some situations...
 - But, what about `Object ArrayList.get(int index)`?
 - Relies on client to check for error
 - Amazing how often this is neglected. E.g. C `malloc()`
 - Does not promote *robust* programs

Error Handling (wrong way)

```
/**  
if n >= 0 return n! else return -1  
*/  
public static int factorial(int n){  
    if(n < 0){ return -1; }//represents an error condition  
    if(n == 0){ return 1; }  
    return n * factorial(n - 1);  
}
```

- Pre-exception solution. Still used in some situations...
 - But, what about `Object ArrayList.get(int index)`?
 - Relies on client to check for error
 - Amazing how often this is neglected. E.g. `C malloc()`
 - Does not promote *robust* programs
 - Error-checking code often horribly entangled with normal code

If there is no good answer

- If there is no good answer, do not make up a random answer.
- If there is no good answer, throw an exception!

Why?

- Invalid answers are normal values and can be stored for a long time before they are involved in execution in some way
- Exceptions have to be handled before execution can resume.

Example:

- You make a `tallestPerson(listPersons)` method returning `null` on empty lists.
- User code calls `tallestPerson(_)` and save the result in a field. Long after that, we get a `NullPointerException` about a field somewhere else in the code, **far from the point in space/time where the error originated!**

Introducing Exceptions

A language construct designed to deal with

- errors
- exceptional behaviour

Sometimes they can be also useful for terminating complex computations.

Differences between:

- **return new** Foo();
- **throw new** Foo();

Introducing Exceptions

```
class MyList{  
    public int size(){...}  
    public Object get(int index){  
        var outBound= index < 0 || index >= size();  
        if (outBound){ throw new ArrayIndexOutOfBoundsException(); }  
        ...// happy path here  
    }  
}
```

Introducing Exceptions

```
class MyList{
    public int size(){...}
    public Object get(int index){
        var outBound= index < 0 || index >= size();
        if (outBound){ throw new ArrayIndexOutOfBoundsException(); }
        ...// happy path here
    }
}
...
MyList v= new MyList();
try { v.get(0); } // error
catch (ArrayIndexOutOfBoundsException e){
    // deal with error
}
```

Introducing Exceptions

```
class MyList{
    public int size(){...}
    public Object get(int index){
        var outBound= index < 0 || index >= size();
        if (outBound){ throw new ArrayIndexOutOfBoundsException(); }
        ...// happy path here
    }
}
...
MyList v= new MyList();
try { v.get(0); } // error
catch (ArrayIndexOutOfBoundsException e){
    // deal with error. Crucially, this can be far away from the .get call
}
```

- Exceptions signal *exceptional behaviour*
 - Method can terminate *normally* by returning result
 - Or *abruptly*, by throwing an exception

Try-Catch Blocks

```
void bar(){ throw new NullPointerException(); }
```

```
void foo(){  
    try { bar(); }  
    catch (IndexOutOfBoundsException e) {  
        System.out.println("foo");  
    }  
}
```

```
void main(...){  
    try { foo(); }  
    catch (NullPointerException e){  
        System.out.println("main");  
    }  
}
```

A: "foo"
B: "main"

Try-Catch Blocks

```
void bar(){ throw new NullPointerException(); }
```

```
void foo(){  
    try { bar(); }  
    catch (IndexOutOfBoundsException e) {  
        System.out.println("foo");  
    }  
}
```

```
void main(...){  
    try { foo(); }  
    catch (NullPointerException e){  
        System.out.println("main");  
    }  
}
```

A: "foo"
B: "main"

- When an Exception is thrown
 - Control passes to enclosing try-catch block which matches exception type

Try-Catch Blocks

```
void bar(){ throw new NullPointerException(); }
```

This try-catch block doesn't match

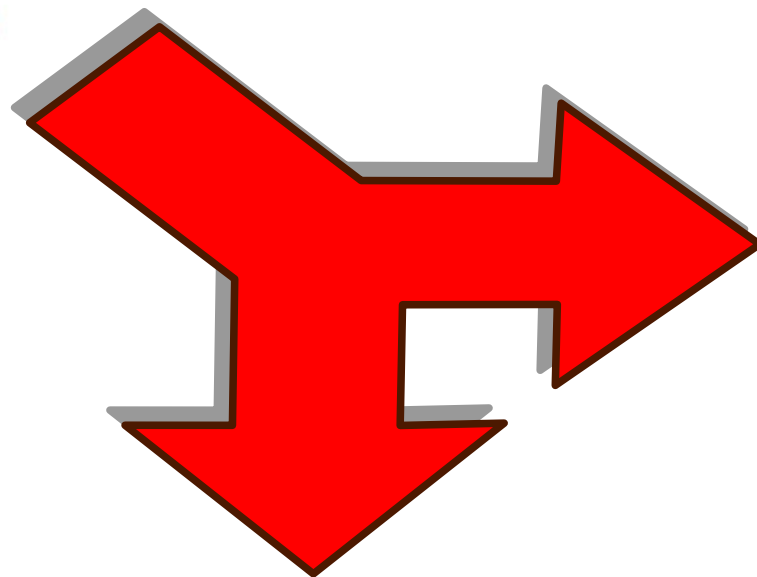
```
void foo(){  
    try { bar(); }  
    catch (IndexOutOfBoundsException e) {  
        System.out.println("foo");  
    }  
}
```

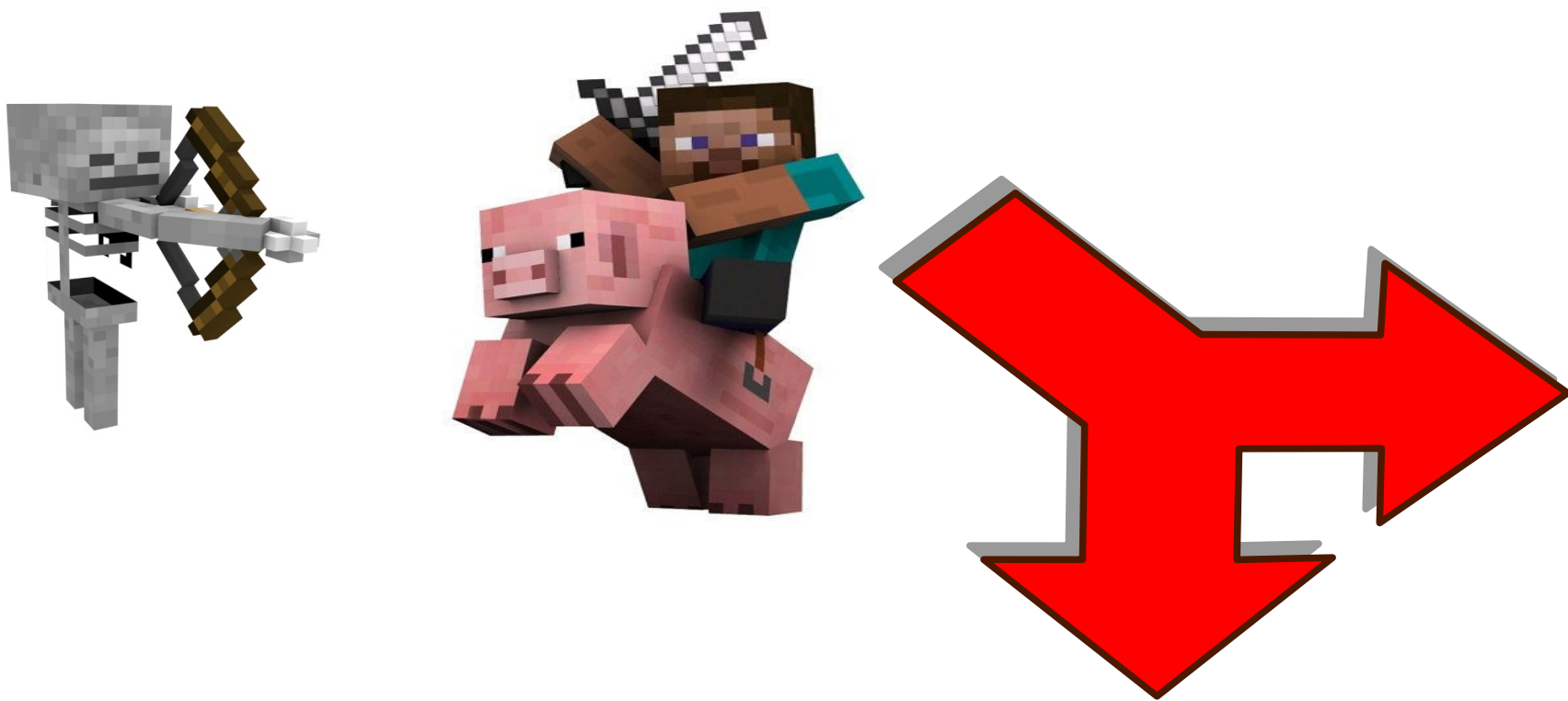
```
void main(...){  
    try { foo(); }  
    catch (NullPointerException e){  
        System.out.println("main");  
    }  
}
```

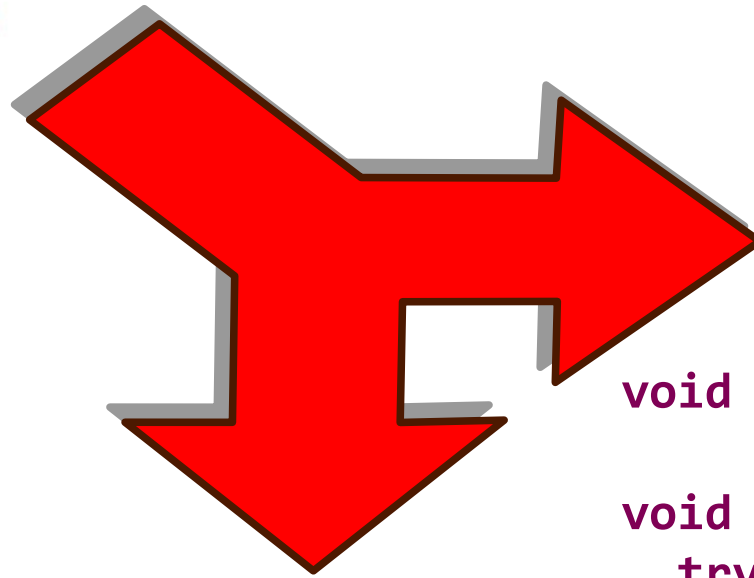
Control transferred here

- When an Exception is thrown
 - Control passes to enclosing try-catch block which matches exception type









```
void bar(){throw new NullPointerException();}
```

```
void foo(){  
    try { bar(); }  
    catch (IndexOutOfBoundsException e) {  
        System.out.println("foo");  
    }  
}
```

```
void main(...){  
    try { foo(); }  
    catch (NullPointerException e){  
        System.out.println("main");  
    }  
}
```



Nesting Exceptions

- Exceptions might have a message and/or a cause associated
- **Methods:** `getCause()`, `getMessage()`, `getStackTrace()`, **etc.**

```
try { lowLevelOp(); }  
catch (LowLevelException e){  
    throw new HighLevelException("my msg", e);  
}
```

- Exceptions are a **language** feature
- Chain of exceptions (`getCause()`) is a **library** feature, that is, a convention programmers use to propagate complex exceptions

Java exceptions and Norse mythology



Java exceptions and Norse mythology

- Odin.
- The father of all gods.
- Throwable
- The father of all kinds of exceptions.
- It is very rare to do `new Throwable(..)`.
- Sometime used as a type.
- Quite rare to do `catch(Throwable t){..}`



Java exceptions and Norse mythology

- Thor.
- The most iconic.
- Exception
- The most iconic kind of Exception, the first that comes to mind.
- Rare to do
 `new Exception(...)`
- `catch(Exception e){..}`
 is often overkill



Java exceptions and Norse mythology

- Loki.
- The sneaky guy
- Error
- The one whose name causes the most dread.
- Common to do
new Error(e)
- catch(Error e){..}
is common in library code



Java exceptions and Norse mythology

- Heimdall: The Unseen Sentinel
- Heimdall's watch, exists without explicit declarations, yet ready to alert and respond when needed.
- RuntimeException
- Helps us to solve limitations of Java by tunneling (Bifröst) errors around the type system.
- Common to do
new RuntimeException(e)
- catch(RuntimeException e){..
is common in library code

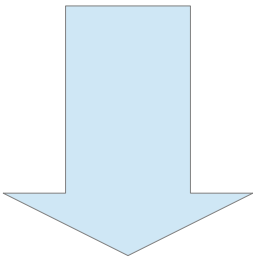
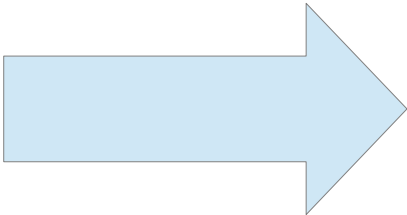
Throwable



Exception



RuntimeException



Error



Family relationships:
Throwable(Odin) is the father (supertype) of

- Error (Loki) and
- Exception (Thor). Exception is the father of
- RuntimeException (Heimdall)

They all spawn their minions

Throwable

Exception

RuntimeException



Checked



Checked



Unchecked

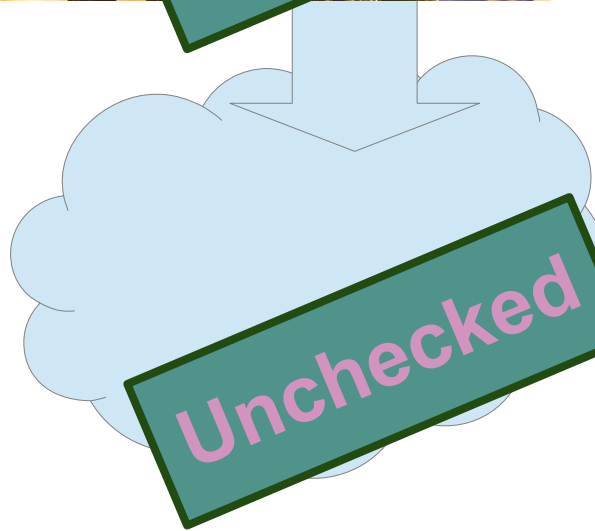
Error



Unchecked



Unchecked



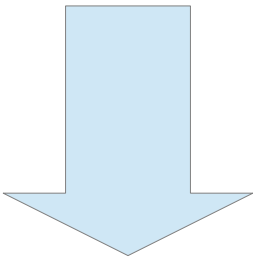
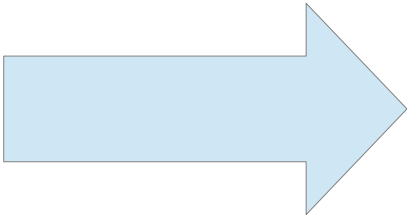
Throwable



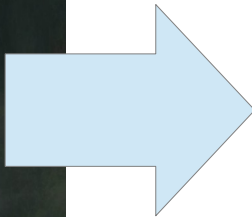
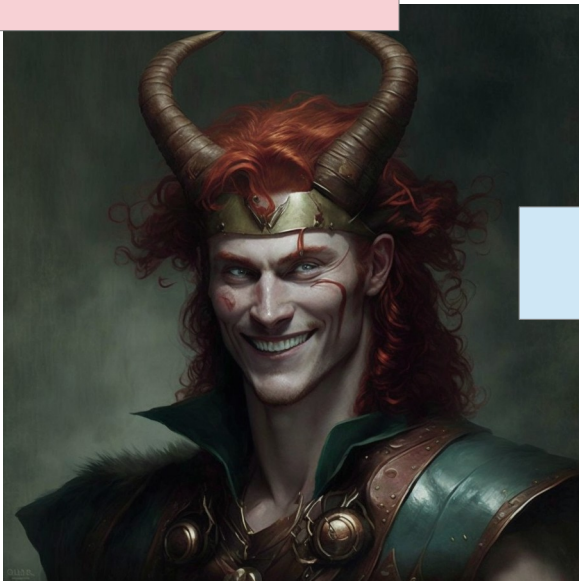
Exception



RuntimeException



Error



An Error is a subclass of Throwable that indicates serious problems that a reasonable application should not try to catch. Most such errors are abnormal conditions.

**AssertionError, AWTError, CoderMalfunctionError, IOException,
LinkageError, ThreadDeath, VirtualMachineError, ...**

Throwable



Exception



RuntimeException



The class Exception and its subclasses are a form of Throwable that indicates conditions that a reasonable application might want to catch.

Error



The class Exception and any subclasses that are not also subclasses of RuntimeException are checked exceptions. Checked exceptions need to be declared in a method or constructor's throws clause if they can be thrown by the execution of the method or constructor and propagate outside the boundary.

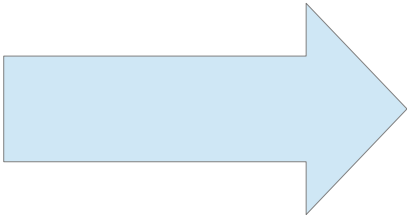
Tons of kids:

- | | | |
|-------------------------------|-------------------------|---------------------------|
| ExecutionException, | FontFormatException, | GeneralSecurityException, |
| InterruptedException, | IOException, | MarshalException, |
| MidiUnavailableException, | MimeTypeParseException, | ParseException, |
| ReflectiveOperationException, | SQLException, | TimeoutException, |
| TooManyListenersException, | XMLParseException, | ... |

Throwable



Exception

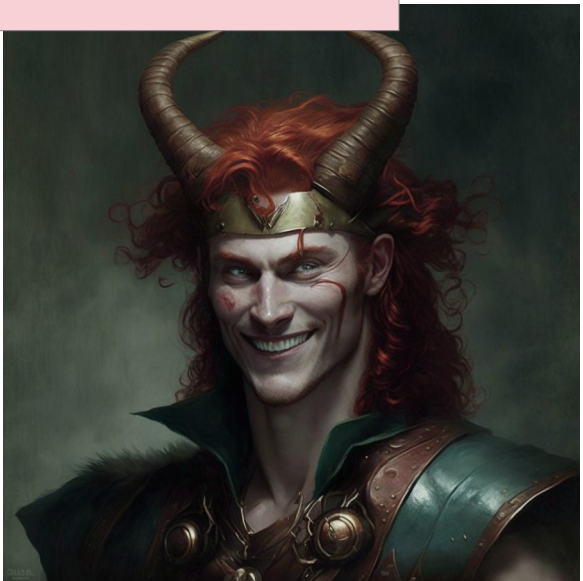


RuntimeException



RuntimeException is the superclass of those exceptions that can be thrown during the normal operation of the Java Virtual Machine.

Error



RuntimeException and its subclasses are unchecked exceptions. Unchecked exceptions do not need to be declared in a method or constructor's throws clause

Tons of kids. Those are actually useful/relevant:
ArithmeticException, ArrayStoreException, ClassCastException, CompletionException, ConcurrentModificationException, EnumConstantNotPresentException, IllegalArgumentException, IndexOutOfBoundsException, NegativeArraySizeException, NoSuchElementException, NullPointerException, SecurityException, UnmodifiableSetException, UnsupportedOperationException,

Resource Handling

```
void foobar(){
    FileOutputStream tmp= new FileOutputStream("tmp.dat");
    try {
        ... // do some complicated computation
        ... // writing results to temporary file
        tmp.close();
        new File("tmp.dat").delete(); // delete temporary file
    }
    catch (IOException e) {
        ... // report write error and return
    }
}
```

- This code has a problem

Finally

```
void foobar(){
    FileOutputStream tmp= new FileOutputStream("tmp.dat");
    try {
        ... // do some complicated computation
        ... // writing results to temporary file
    }
    catch (IOException e) {
        ... // report write error and return
    }
    finally {
        tmp.close();
        new File("tmp.dat").delete(); // delete temporary file
    }
}
```

- Finally clause:
 - Gets executed regardless of how try-block exited
 - E.g. by normal execution, by caught exception or uncaught exception
 - Useful for “cleaning up” allocated resources

Try with resource

```
try (var tmp= new FileOutputStream("out.dat")){//newer, better solution
    ... // do some complicated computation
    ... // writing results to file
}
catch(IOException e) {
    ... // report write error and return
}
```

- From Java7, is a compact and more elegant solution to close resources
 - Convenient alternative to finally
 - Any object that implements `java.lang.AutoCloseable` (includes `java.io.Closeable`) can be used as a resource
 - Not as flexible; for example here we do not delete our file, we only close it.

More metaphors: throw something

- A monster rises from the darkness.
- It will destroy everything onto his path.
- This is representing stack unwinding:
The exception consumes the stack until it is catch-ed.



More metaphors: catch something

- The catch block is like a hero trying to stop the monster.
- It can go both ways.
- If the hero is too weak, the monster will win and the hero will be defeated!



More metaphors: catch something

- If the hero is too weak, the monster will win and the hero will be defeated.
- However, with the 'finally' block, the hero can say their last words!
- **No matter if they win or lose**, they will say those words before the stack unwinds away from them, sending them back into oblivion.



More metaphors: catch something

- If the hero is too weak, the monster will win and the hero will be defeated.
- However, with the 'finally' block, the hero can say their last words!
- **No matter if they win or lose**, they will say those words before the stack unwinds away from them, sending them back into oblivion.

Defend the
main loop!



More metaphors: catch something

- The catch block is like a hero trying to stop the monster.
- It can go both ways.
- If the hero is too strong, the hero will squash the monster,...



More metaphors: catch something

- The catch block is like a hero trying to stop the monster.
- It can go both ways.
- If the hero is too strong, the hero will squash the monster, together with a few city blocks.
(too much collateral damage)



More metaphors: catch something

- We need a Goldilocks hero
- The catch block should use a hero that is 'just right'
- Not too strong
- Not too weak

Otherwise we would capture more than we intend to
== collateral damage



What is the difference?

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
/*c*/
```

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
finally { /*c*/ }
```

Write down a minimal example of code where the presence of finally produces a different behavior!

```
try {  
  
}  
catch (SomeException e){  
  
}  
finally {  
  
}
```

```
try {  
  
}  
catch (SomeException e){  
  
}
```

Is this a valid solution?

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
/*c*/
```

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
finally { /*c*/ }
```

A: YES
B: NO

```
try {  
    throw new SomeException();  
}  
catch (SomeException e){  
    System.out.println("Hello");  
}  
finally {  
    System.out.println("World");  
}
```

```
try {  
    throw new SomeException();  
}  
catch (SomeException e){  
    System.out.println("Hello");  
}  
  
System.out.println("World");
```

Is this a valid solution?

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
/*c*/
```

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
finally { /*c*/ }
```

A: YES
B: NO

```
try {  
    throw new AnotherException();  
}  
catch (SomeException e){  
    System.out.println("Hello");  
}  
finally {  
    System.out.println("World");  
}
```

```
try {  
    throw new AnotherException();  
}  
catch (SomeException e){  
    System.out.println("Hello");  
}  
  
System.out.println("World");
```

Is this a valid solution?

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
/*c*/
```

```
try { /*a*/ }  
catch (SomeException e){ /*b*/ }  
finally { /*c*/ }
```

A: YES
B: NO

```
try {  
    throw new SomeException();  
}  
catch (SomeException e){  
    throw e;  
}  
finally {  
    System.out.println("World");  
}
```

```
try {  
    throw new SomeException();  
}  
catch (SomeException e){  
    throw e;  
}  
  
System.out.println("World");
```

Again: Types of Exception

- Unchecked Exceptions

Subclasses of `RuntimeException` and `Error`

- E.g. `NullPointerException` and `IndexOutOfBoundsException`

- Checked Exceptions

- Subclasses of `Exception`, but not `RuntimeException`

- e.g. `IOException`

- Must be declared in a method's **throws clause**:

- If it throws one, or doesn't catch one thrown by called method
- Otherwise compile-time error

Checked Exceptions

```
void foo() throws IOException {  
    ...  
    throw new IOException("...");  
}
```



```
void bar(){ foo(); }
```

//Compile time error



```
void bar() throws IOException { foo(); } //OK
```



```
void bar(){  
    try { foo(); }  
    catch (IOException e){...}  
}
```

//OK

Checked Exceptions

- Why checked exceptions?
 - Signal recoverable problems
 - E.g. `FileNotFoundException` (in an interactive program) **versus** `NullPointerException` **or** `ArithmeticException`
 - Programs can **typically recover** from such errors
 - ***Force*** clients to deal with problem
 - Programmer cannot ignore possible error
 - Compile time errors are better than runtime errors!

Checked vs Unchecked

- Checked exceptions:
 - Signal recoverable problems / expected behaviour
- Unchecked exceptions:
 - Observed bug (not always a fatal one)

```

class MyCheckedException extends Exception{

    MyCheckedException(){ super(); }
    MyCheckedException(String m){ super(m); }
    MyCheckedException(String m, Throwable c){ super(m,c); }
}

```

Deal with
Checked
exceptions

```

...
try {...}
catch (MyCheckedException e){
    /*provide alternative behaviour*/
}

```

```

class MyUncheckedException extends RuntimeException{

    MyUncheckedException(){super();}
    MyUncheckedException(String m){super(m);}
    MyUncheckedException(String m, Throwable c){ super(m,c);}
}

```

Deal with
Unchecked
exceptions

```

...
try {...}
catch (MyUncheckedException e){
    /*resume the execution. In older java usually only used in the
    artisan/tool (ex master/slave) pattern.
    Used in more situations in the new Java style*/
}

```

Turn Checked Exceptions into Errors

- Checked exceptions means there is still (potential) hope for a solution
 - Sometimes, you know there is no hope
 - You know something just should not happen.
- Scenario (this really happened)
 - Designed and built “Simple Program Interpreter”
 - In first version of language no InputStatement
 - Statements don’t declare “**throws** IOException”
 - Added InputStatement, which reads input!
 - But, InputStream.read() throws IOException
 - What to do?

Problems with Checked Exceptions

- This is the situation:

```
interface Statement{  
    void execute();  
}  
class InputStatement implements Statement{  
    public void execute(){  
        InputStream input= ...;  
        input.read(); //throws IOException  
    }  
}
```

- Options:
 - Declare Statement.execute() (+ all subclasses) throws IOException
 - Deal with Exception in InputStatement somehow

What not to do!

```
interface Statement{
    void execute();
}
class InputStatement implements Statement{
    public void execute(){
        InputStream input= ...;
        try { input.read(); }//throws IOException
        catch (Exception e){}
    }
}
```



- All exceptions (including `RuntimeException`) are “swallowed”
- This is equivalent of calling a hero too strong for the monster too much unneeded collateral damage!

What not to do!

```
interface Statement{
    void execute();
}
class InputStatement implements Statement{
    public void execute(){
        InputStream input= ...;
        try { input.read(); }//throws IOException
        catch (IOException e){}
    }
}
```

IOException is still being “swallowed”

What not to do!

```
interface Statement{
    void execute();
}
class InputStatement implements Statement{
    public void execute(){
        InputStream input= ...;
        try { input.read(); }//throws IOException
        catch (IOException e){
            e.printStackTrace();
        }
    }
}
```

- `IOException` written on the “standard output”; and than “swallowed”!
- Sadly, this is the eclipse “template”

This is equivalent of the application faking their own death before going to hide happily in a tropical paradise.



YES! The right thing to do: Turn Checked Exceptions into Errors

```
interface Statement{
    void execute();
}
class InputStatement implements Statement{
    public void execute(){
        InputStream input= ...;
        try { input.read(); } //throws IOException
        catch (IOException e){ throw new Error(e); }
    }
}
```

- Exception **rethrown** as unchecked exception!
- Just mark the error as being an observed bug.
- (AssertionError would work too...)

